

**Rapport de Travail Pratique :**  
**Algorithme et programmation**

# **Prise en main du langage C**

---

**Effectué par:**

CARREIRA Clément

BERNARD Alexis

E3FI / 1I

**A l'attention de :**

M. Romain NEGREL

Enseignants de l'unité Algorithme et programmation

**Etablissement :** ESIEE Paris - Cité Descartes, 2 Bd Blaise Pascal, 93160 Noisy-le-Grand

**Période de réalisation :** du 18/03/2026 au 27/03/2026

**Année :** 2026

# Sommaire

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| <b>Sommaire.....</b>                                                    | <b>2</b>  |
| <b>Introduction.....</b>                                                | <b>3</b>  |
| <b>1. Un premier programme C sous LINUX.....</b>                        | <b>3</b>  |
| 1.1 Prise en main de linux et du langage C.....                         | 3         |
| 1.2 Exemple de programme avec la fonction factorielle.....              | 4         |
| <b>2. Les types du langage C.....</b>                                   | <b>6</b>  |
| Cas 2 : valeurs proches autour de 1 tolérance relative est adaptée..... | 10        |
| 1.0 et 1.0+1e-10.....                                                   | 10        |
| Cas 3 : très grandes valeurs.....                                       | 10        |
| <b>3. Manipulation de pointeurs.....</b>                                | <b>11</b> |
| <b>4. Interaction avec des tableaux.....</b>                            | <b>12</b> |

# Introduction

Ce rapport a pour objectif de présenter les travaux effectués lors des 2 premières séances de travaux pratiques de l'unité **Algorithme et programmation pour l'ingénieur** dirigé par M. NEGREL. Ces séances nous ont permis de prendre en main le langage de programmation C et d'acquérir de nouvelles connaissances dans le domaine de la manipulation de pointeurs et la gestion de la mémoire d'un programme. Nous commencerons par vous présenter un premier programme C dans l'environnement linux, ensuite une exploration des différents type de ce langage, en troisième partie la manipulation de pointeurs et d'adresses mémoires et enfin une manipulation de tableaux en C.

## 1. Un premier programme C sous LINUX

### 1.1 Prise en main de linux et du langage C

- a) Dans un premier temps nous créons le répertoire de travail grâce aux commandes linux

```
Algorithmme-ProgrammationS3 on ~ main [!?]
❯ mkdir TP1
mkdir: cannot create directory 'TP1': File exists
(base)
Algorithmme-ProgrammationS3 on ~ main [!?]
❯ cd TP1
(base)
Algorithmme-ProgrammationS3/TP1 on ~ main [!?] via C v13.3.0-gcc
❯ |
```

- b) Ensuite nous ouvrons le programme dans vscode

```
Algorithmme-ProgrammationS3/TP1 on ~ main [!?] via C v13.3.0-gcc
❯ code pgm01.c
(base)
```

- c) Après avoir installer **gcc** nous compilons le code c

```

Algorithmme-ProgrammationS3/TP1 on 1 main [!?] via C v13.3.0-gcc
❯ gcc pgm01.c -o pgm01
(base)
Algorithmme-ProgrammationS3/TP1 on 1 main [!?] via C v13.3.0-gcc
❯ |

```

d) Le programme compile sans erreurs, nous pouvons ensuite l'exécuter

```

Algorithmme-ProgrammationS3/TP1 on 1 main [!?] via C v13.3.0-gcc
❯ ./pgm01
Entrez un nombre entier : 5
5 x 5 = 25
(base)
Algorithmme-ProgrammationS3/TP1 on 1 main [!?] via C v13.3.0-gcc took 3s
❯ |

```

e) Nous compilons le programme en y ajoutant des paramètres supplémentaires :

- Wall : active un ensemble courant d'avertissements
- Wextra : active des avertissements supplémentaires
- std=c11 : demande l'utilisation de la norme C11 ;

```

Algorithmme-ProgrammationS3/TP1 on 1 main via C v13.3.0-gcc
❯ gcc -Wall -Wextra -std=c11 pgm01.c -o pgm01
(base)
Algorithmme-ProgrammationS3/TP1 on 1 main via C v13.3.0-gcc
❯ |

```

1: Nous observons pas d'erreur ni de warning lors de la compilation avec ces nouveaux paramètres

3: Pour voir les potentielles erreurs et warning afin de les corriger avant la compilation final

## 1.2 Exemple de programme avec la fonction factorielle

Le code source du programme de la fonction factorielle nous a été fourni par M. NEGREL, nous avons dans un premier temps analyser et exécuter le code afin de mieux comprendre

le fonctionnement de celui-ci. Voici les résultats obtenu lors de l'exécution du programme :

```
clement@PC-Portable:~/algo/ESPI - Algorithmique et programmation pour l'ingenieur - IPI$ ./factorielle
Entrez un entier positif.
0 : terminer
-1 : afficher les factorielles de 1 a 15
Votre choix : 1
factorielle_iterative( 1) =      1
factorielle_recursive( 1) =      1

Entrez un entier positif.
0 : terminer
-1 : afficher les factorielles de 1 a 15
Votre choix : 2
factorielle_iterative( 2) =      2
factorielle_recursive( 2) =      2

Entrez un entier positif.
0 : terminer
-1 : afficher les factorielles de 1 a 15
Votre choix : 5
factorielle_iterative( 5) =     120
factorielle_recursive( 5) =     120

Entrez un entier positif.
0 : terminer
-1 : afficher les factorielles de 1 a 15
Votre choix : 10
factorielle_iterative(10) =    3628800
factorielle_recursive(10) =    3628800
```

```
Entrez un entier positif.
0 : terminer
-1 : afficher les factorielles de 1 a 15
Votre choix : -1
n | factorielle_iterative | factorielle_recursive
---+-----+-----
1 | 1 | 1
2 | 2 | 2
3 | 6 | 6
4 | 24 | 24
5 | 120 | 120
6 | 720 | 720
7 | 5040 | 5040
8 | 40320 | 40320
9 | 362880 | 362880
10 | 3628800 | 3628800
11 | 39916800 | 39916800
12 | 479001600 | 479001600
13 | 1932053504 | 1932053504
14 | 1278945280 | 1278945280
15 | 2004310016 | 2004310016
```

On peut déjà constater une erreur dans le programme en fonction des résultats obtenus, à la place d'une croissance exponentielle propre à la fonction factorielle, nous obtenons une croissance plus logarithmique lorsque l'on regarde attentivement les valeurs de 1 à 15. Cela est dû à la taille allouée pour le type de donnée **int** qui est dépassé par les valeurs et donc, il n'affiche qu'une certaine quantité de chiffres du nombre calculé.

Pour pallier cela, on va nous allons ajouter une variable temporaire de type **long**, un type de donnée conçu également pour les entiers mais avec une plus grosse capacité. On affecte cette variable au résultat calculé à l'étape *n* de notre calcul du factorielle, ensuite nous comparons le résultat avec la plus grande valeur possible contenu dans un **int** nommé *INT\_MAX*. Si notre résultat est effectivement plus grand, nous renvoyons la valeur -1, sinon, nous continuons le calcul. Voici le code final de la fonction **itérative** :

```
int factorielle_iterative(int n) {
    int i, p = 1;
    for (i = 1; i <= n; ++i){
        long tmp = p;
        tmp *= i;
        if (tmp > INT_MAX){
            return -1;
        }
        p = tmp;
    }
    return p;
}
```

Pour la version récursive, nous utilisons une méthode légèrement différente, nous calculons dans l'autre sens. Pour traduire simplement, on vérifie si la fonction factorielle est assez petite pour les valeurs de 0 à n (avec n la valeur du factorielle que l'on veut) afin de le contenir dans un `int`, sinon on renvoie -1.

```
int factorielle_recurive(int n) {
    if (n <= 0)
        return 1;
    else if (factorielle_recurive(n - 1) > INT_MAX / n)
        return -1;
    else if (factorielle_recurive(n - 1) < 0)
        return -1;
    else
        return n * factorielle_recurive(n - 1);
}
```

Voici le résultat obtenu :

```
clement@PC-Portable:~/algo/E3FI - Algorithmique et programmation pour
l'ingénieur - TP1$ ./factorielle

Entrez un entier positif.
0 : terminer
-1 : afficher les factorielles de 1 a 15
Votre choix : -1
n | factorielle_iterative | factorielle_recurive
---|---|---
1 | 1 | 1
2 | 2 | 2
3 | 6 | 6
4 | 24 | 24
5 | 120 | 120
6 | 720 | 720
7 | 5040 | 5040
8 | 40320 | 40320
9 | 362880 | 362880
10 | 3628800 | 3628800
11 | 39916800 | 39916800
12 | 479001600 | 479001600
13 | -1 | -1
14 | -1 | -1
15 | -1 | -1

Entrez un entier positif.
0 : terminer
-1 : afficher les factorielles de 1 a 15
Votre choix :
```

## 2. Les types du langage C

### 1) Détermination de type

| Type de données | Types | Justification |
|-----------------|-------|---------------|
|-----------------|-------|---------------|

|                                |        |                                                                                                                             |
|--------------------------------|--------|-----------------------------------------------------------------------------------------------------------------------------|
| Année de naissance             | int    | Car il s'agit d'un nombre entier et qu'il est possible des calculs dessus                                                   |
| Numéro de téléphone            | char[] | On l'utilise car on peut contenir des zéros au début pas utilisés pour les calculs et parfois des symboles (+33).           |
| Numéro de département français | char[] | Comme certains départements contiennent des lettres comme la corse donc un type numérique n'est pas forcément préférable    |
| Taux de TVA                    | double | On peut utiliser un double les taux de TVA peuvent être en décimal.                                                         |
| Taille de fichier              | long   | Il ne s'agit pas d'un nombre décimal mais peut être parfois très grand pour certain fichier int serait parfois insuffisant. |
| Numéro de chambre d'hôtel      | int    | Nombre entier car il s'agit d'un nombre pas nécessairement grand                                                            |
| Note d'examen sur 20           | double | Elle peut être décimal                                                                                                      |
| Adresse email                  | char[] | Une adresse mail peut contenir des lettres, chiffres,                                                                       |

|                                       |        |                                                            |
|---------------------------------------|--------|------------------------------------------------------------|
|                                       |        | symboles donc besoin d'un char[]                           |
| Température en degrés Celsius         | double | Peut être décimale et négative.                            |
| Nombre d'étudiants dans une promotion | int    | Nombre entier, généralement pas dans une plage raisonnable |

2) Code des quatres fonction de comparaison :

```

/*
 * TODO
 * Retourner un entier non nul si
 *  $|a-b| \leq \max(\text{rel\_tol} * \max(|a|, |b|), \text{abs\_tol}), 0$  sinon.
 */
int proche_pep_0485(double a, double b, double rel_tol, double abs_tol) {
    if (fabs(a - b) <= MAX(rel_tol * MAX(fabs(a), fabs(b)), abs_tol)) {
        return 1;
    }
    else {
        return 0;
    }
}

```



```

Algorithmme-ProgrammationS3/TP1 on 17 main [?] via C v13.3.0-gcc 17 18
> ./comparaison_double
=== Cas 1 : meme valeur mathematique, calcul flottant different ===
a = 0.30000000000000004, b = 0.29999999999999999
equal : 0
proche_abs : 1 (eps = 9.999999999999998e-13)
proche_rel : 1 (eps = 9.999999999999998e-13)
proche_pep_0485 : 1 (rel_tol = 1.0000000000000001e-09, abs_tol = 0)

=== Cas 2 : valeurs proches autour de 1 ===
a = 1, b = 1.0000000001
equal : 0
proche_abs : 0 (eps = 9.999999999999998e-13)
proche_rel : 1 (eps = 1.0000000000000001e-09)
proche_pep_0485 : 1 (rel_tol = 1.0000000000000001e-09, abs_tol = 0)

=== Cas 3 : valeurs proches mais tres grandes ===
a = 1000000000000, b = 1000000000001
equal : 0
proche_abs : 0 (eps = 1.0000000000000001e-09)
proche_rel : 1 (eps = 1.0000000000000001e-09)
proche_pep_0485 : 1 (rel_tol = 1.0000000000000001e-09, abs_tol = 0)

=== Cas 4 : valeurs tres proches de zero ===
a = 9.999999999999998e-13, b = 2e-12
equal : 0
proche_abs : 1 (eps = 9.999999999999998e-13)
proche_rel : 0 (eps = 1.0000000000000001e-09)
proche_pep_0485 : 0 (rel_tol = 1.0000000000000001e-09, abs_tol = 0)

```

```

=== Cas 5 : proche de zero avec tolerance absolue utile ===
a = 9.999999999999998e-13, b = 2e-12
equal : 0
proche_abs : 1 (eps = 9.999999999999998e-13)
proche_rel : 0 (eps = 1.0000000000000001e-09)
proche_pep_0485 : 1 (rel_tol = 1.0000000000000001e-09, abs_tol = 9.999999999999998e-13)

=== Cas 6 : zero et tres petite valeur ===
a = 0, b = 1.0000000000000001e-15
equal : 0
proche_abs : 1 (eps = 9.999999999999998e-13)
proche_rel : 0 (eps = 1.0000000000000001e-09)
proche_pep_0485 : 0 (rel_tol = 1.0000000000000001e-09, abs_tol = 0)

=== Cas 7 : zero et tres petite valeur avec abs_tol ===
a = 0, b = 1.0000000000000001e-15
equal : 0
proche_abs : 1 (eps = 9.999999999999998e-13)
proche_rel : 0 (eps = 1.0000000000000001e-09)
proche_pep_0485 : 1 (rel_tol = 1.0000000000000001e-09, abs_tol = 9.999999999999998e-13)

=== Cas 8 : influence de rel_tol ===
a = 1000, b = 1000.1
equal : 0
proche_abs : 0 (eps = 0.001)
proche_rel : 0 (eps = 9.999999999999995e-07)
proche_pep_0485 : 0 (rel_tol = 9.999999999999995e-07, abs_tol = 0)

a = 1000, b = 1000.1
equal : 0
proche_abs : 0 (eps = 0.001)
proche_rel : 1 (eps = 0.0001)
proche_pep_0485 : 1 (rel_tol = 0.0001, abs_tol = 0)

```

**Cas 1 : Égalité stricte est trop sévère :**

- $0.1 + 0.2$  donne  $0.30000000000000004$
- $0.3$  est représenté par  $0.29999999999999999$

Mathématiquement, c'est la même valeur, mais en c les représentations binaires diffèrent légèrement.

#### **Cas 4 la tolérance absolue est adaptée :**

-a =  $1e-12$ , b =  $2e-12$

-différence :  $1e-12$

Avec  $\text{eps\_abs} = 1e-12$  la fonction `proche_abs` renvoie 1.

Ici, la différence est minuscule en valeur absolue. le cas est aussi valable pour le test numéro 6

-a = 0

-b =  $1e-15$

La différence est très petite.

Avec  $\text{eps\_abs} = 1e-12$  la fonction `proche_abs` renvoie encore 1.

#### **Cas 2 : valeurs proches autour de 1 tolérance relative est adaptée**

1.0 et  $1.0+1e-10$

-différence :  $1e-10$

La tolérance absolue avec  $1e-12$  est trop stricte donc `proche_abs` = 0.

Mais relativement à une valeur de l'ordre de 1,  $1e-10$  est un très petit écart, donc `proche_rel` = 1.

#### **Cas 3 : très grandes valeurs**

- $1e12$  et  $1e12 + 1$

- différence absolue : 1

En absolu, 1 est bien plus grand que  $1e-9$ , donc `proche_abs = 0`.

Mais relativement à  $1e12$ , un écart de 1 est infime :

Donc `proche_rel = 1`.

Utilisation de `rel_tol + abs_tol`

L'égalité stricte `a == b` est trop sévère pour les nombres flottants, car deux calculs mathématiquement équivalents peuvent donner des représentations binaires légèrement différentes, comme dans le cas  $0.1 + 0.2$  comparé à  $0.3$ .

La tolérance absolue `|a-b| <= eps` est adaptée lorsque l'on veut borner directement l'écart numérique, en particulier pour des valeurs très proches de zéro.

La tolérance relative `|a-b| <= eps * max(|a|, |b|)` est adaptée lorsque l'on veut juger l'écart proportionnellement à la grandeur des valeurs, par exemple pour des nombres très grands.

Enfin, la combinaison `max(rel_tol * max(|a|, |b|), abs_tol)` est la plus complète, car elle utilise une tolérance relative pour les valeurs éloignées de zéro et une tolérance absolue pour les valeurs très petites, ce qui couvre des cas complémentaires.

- 3) On utilise un cast de pointeur car avec cette méthode on interprète l'adresse d'une variable de type double comme un pointeur vers `unsigned char`. En effectuant ce cast de pointeur, on accède directement aux octets après il faut ensuite copier ces 8 octets dans un tableau de type `unsigned char[8]`.

### 3. Manipulation de pointeurs

Dans cette section, nous allons apprendre à manipuler des pointeurs, notamment avec des fonctions d'échange de valeurs via des effets de bords. Il nous a été demandé de compléter 2 fonctions d'échange permettant d'échanger les valeurs des 2 paramètres envoyés dans les fonctions. Or, une fonction utilise les pointeurs et l'autre ne les utilise pas. La fonction qui n'utilise pas de pointeurs ne fonctionne pas pour une raison particulière : lorsqu'une fonction est appelée, elle copie les paramètres envoyés dans sa zone dans la pile. Si nous lui envoyons directement des valeurs, elle ne peut pas modifier les variables déclarées en dehors de sa zone mémoire, mais si nous lui envoyons un pointeur vers notre valeurs, alors elle va copier l'adresse mémoire et nous pourrons alors accéder à notre valeur et la modifier. Le fait de modifier des valeurs sans les renvoyer via l'instruction `return` se nomme effet de bord.

## 4. Interaction avec des tableaux

Tout d'abord, un tableau en C est en réalité un pointeur pointant vers la première valeur de celui-ci. Si nous voulons le  $n$ ème élément de ce tableau nous devons incrémenter le pointeur de  $n$ .

Nous avons commencé par un cas classique : le calcul d'une moyenne des éléments d'un tableau. Nous avons besoin du nombre d'éléments du tableau et du tableau en question. L'algorithme est classique, on parcourt le tableau jusqu'à la fin, on fait la somme de tous les éléments et on la divise par le nombre d'élément.

Par la suite, nous avons créé des algorithmes récursifs permettant de reconnaître si une phrase contient ou commence par une certaine chaîne de caractères en considérant les chaînes de caractères comme des tableaux de **char** se terminant par le caractère **'\0'**.